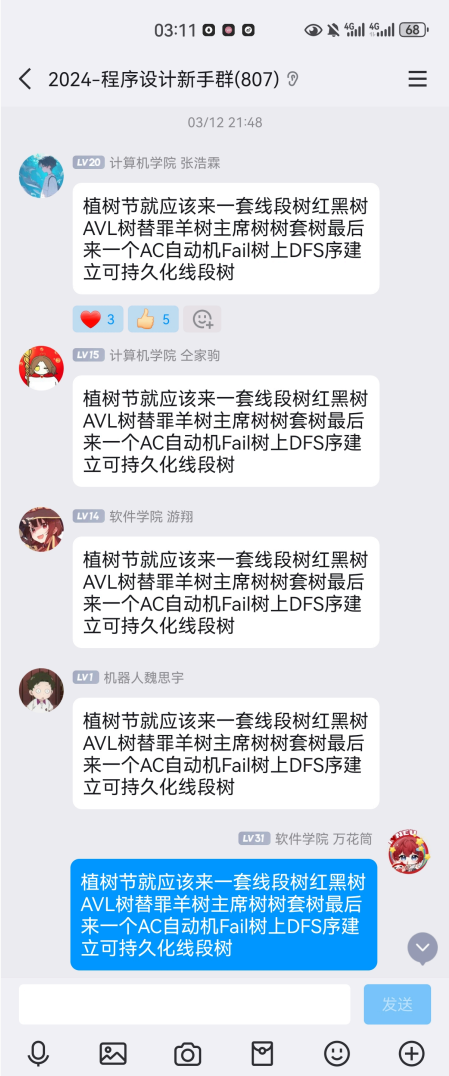
树论

从入门到入土

Intro

Intro

曾几何时植树节，新手群出现了这样一段刷屏...
你放心，大家刷的，今天全都不讲！
大家刷屏的树，都是树形结构的数据结构。
本专题我们就聚焦于“树形结构”，来好好研究研究“树论”。



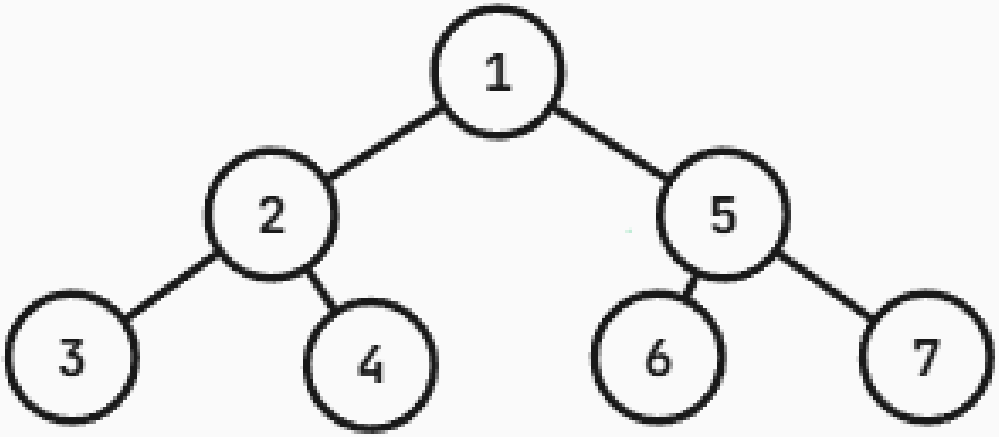
Intro

普通人眼中的树 BE LIKE

程序员眼中的树 BE LIKE

Intro

就这样，“树”诞生了！



树基础

什么是树

在离散数学、数据结构、算法竞赛等领域，对树常有以下几种等价的形式化定义：

- n 个结点，$n-1$ 条边的连通无向图。
- 无向无环的连通图。
- 任意两个结点之间有且仅有一条简单路径的无向图。
- 任何边均为桥（删掉此边后图不再连通）的连通图。
- 没有环，并且在任意两个结点之间添加一条边之后得到的图含有唯一一个环的图。

无根树和有根树

- 无根树：一个没有指定根的树叫做无根树。
- 有根树：在无根树的基础上，指定一个结点作为树的根，这棵树就形成了一棵有根树。
- 由于有根树有很多便于维护的信息，以及遍历顺序，因此无根树的问题通常也是转化为有根树来解决。

Tips

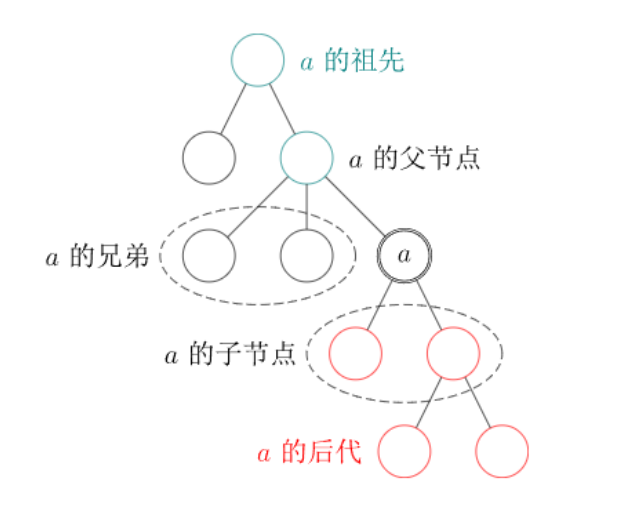
无论是有根树还是无根树，通常都是无向图，因为我们并没有指定边的方向。
即使是有根树，也只是确定了一些父子关系，但由于常见的树上问题的特殊性，边一般都是双向的。
但有时候题目或许会自行定义带有方向的边，这就具体问题具体分析了。

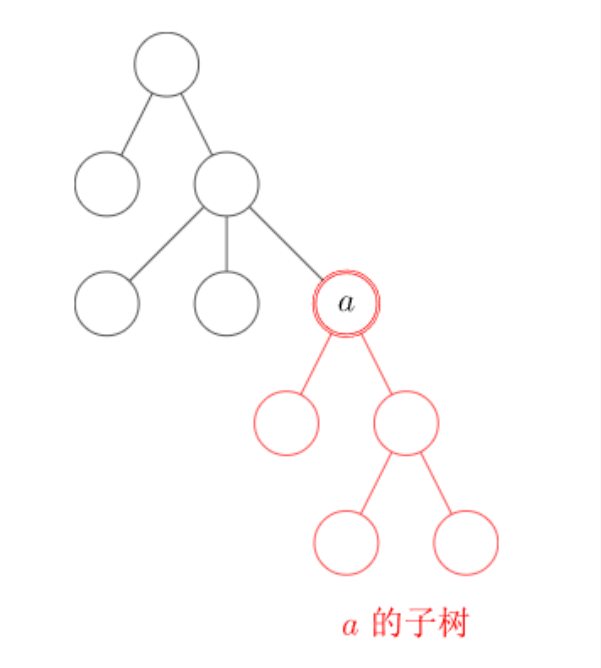
相关概念定义

以下概念，均是在有根树下定义的：

- 结点深度：一个结点到根结点的边的数量。
- 父结点：除了根节点之外，和这个结点直接相连的层数低一层的结点。
- 祖先结点：一个结点到根的路径上，除了它本身的结点。
- 兄弟结点：同一个父亲的多个子结点互为兄弟。
- 子树：删掉与父结点的连边后，该结点所在的子图。

相关概念定义



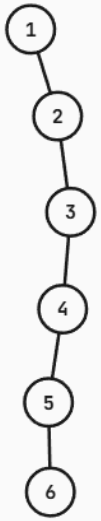


特殊的树

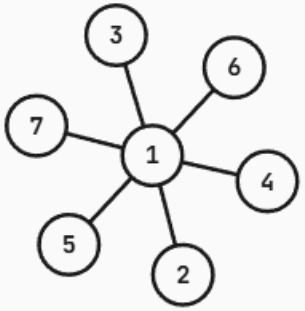
一些特殊的树可以用于分析一些树上问题的极限复杂度，避免 TLE 或是在 TLE 时进行合理判断。

- 链：很好理解，两两相连形成一条链，像数组一样。
- 菊花图：一个点作为根，其它所有结点直接连向这个点，也可用于分析一些问题的时间复杂度。
- 二叉树：树上问题其实很少专门考察二叉树，因为二叉树问题常常也可以像一般的树上问题那么分析。二叉树与其说是一个图，更像是一个分治思想的运用，许多数据结构都是二叉树结构，如线段树、红黑树。

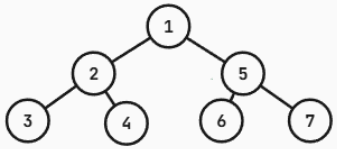
特殊的树



链



菊花图



二叉树

树的特殊性

树是一种特殊的图，但其实树不仅仅可以看作一种特殊的图。
前面提到过，树上无环，任何两个结点都只有一条唯一的简单路径。
因此，我们还可以把树看作是很多个数组的拼接，任何两个结点之间的路径都可以看作一个数组。
所以，树其实还可以看作是对数组这样的线性空间的一个扩展，几乎任何可以在数组上出的题，都可以在出在树上。
也正是因此，树虽然是一种特殊的图，但是树上的问题和图上的问题在考察方向、思考方向、解决方式上，有着很大的差异性，这也正是树能够在算法竞赛中独占一个板块并且几乎每场比赛都会有一个树相关的题目的原因。

存图方式

树就是一种特殊的图，所以，存储一棵树，像图那样，按照自己喜欢的方式存储就好，用二维 vector 或是链式前向星。
个人喜欢用二维 vector，后面相关的代码部分的图均为使用二维 vector 存储。

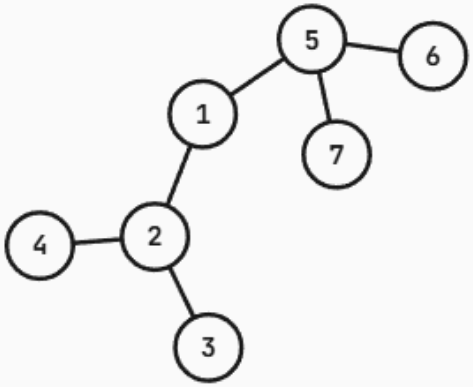
树的遍历

DFS - 深度优先搜索

深度优先搜索

故名思意，优先搜索更深的结点，“不撞南墙不回头”。
先访问根节点，然后分别访问根节点每个儿子的子树。
可以用来求出每个节点的深度、父亲等信息。

DFS - 深度优先搜索



比如在上图，如果我们以 5 为根，就可以有下面这个 DFS 遍历顺序：

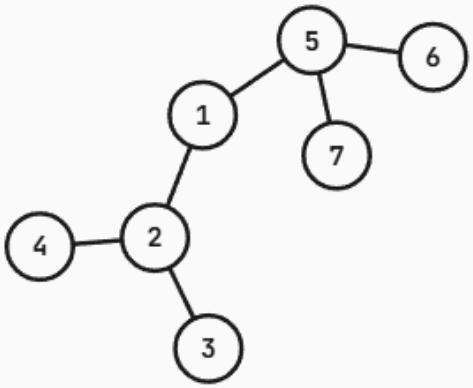
$$5, 1, 2, 4, 3, 7, 6$$

同一棵树有多种不同不同的 DFS 遍历顺序，比如上图还可以是：

$$5, 6, 7, 1, 2, 4, 3$$

这是因为我们只规定了深度优先，而兄弟结点的先后遍历是随意的。

DFS - DFS 序



按照 DFS 的顺序搜索得到的首次访问的结点序列称为 DFS 序，每个结点在 DFS 序中的位置称为其在 DFS 序中的编号（或时间戳）。
比如：上图的 DFS 序可以是：5, 1, 2, 4, 3, 7, 6。
各个结点（从 1 到 7）的 DFS 序编号依次为：2, 3, 5, 4, 1, 7, 6。

DFS - 特点

DFS 的特点

树上的 DFS 有一个特点，对于一个子树，遍历到该子树的根节点之后，会把其子树遍历完全，才会去遍历该子树外的结点，所以很适合用 DFS 把一些信息自底向上维护，因此很多树上问题都是在进行 DFS 的同时解决的，比如树形 DP。

DFS - 常见代码写法

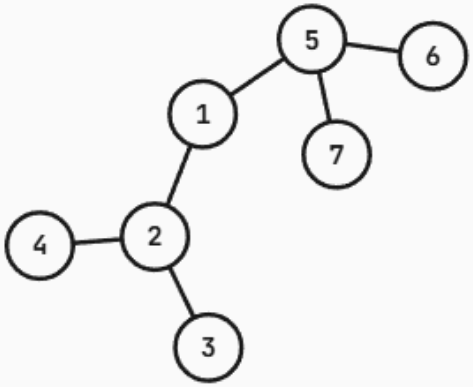
```
// u为当前结点，fa为当前结点的父结点
void dfs(int u, int fa) {
    // 处理当前结点在继续搜索之前要处理的信息

    // DFS主体，向下搜索
    for(auto &x : g[u]) {
        if(x == fa) {
            continue;
        }

        dfs(x, u);
    }

    // 处理当前子树搜索完后要处理的信息
}
```

BFS - 宽度优先搜索

宽度优先搜索

和一般的图的 BFS 并无区别，从根节点开始，一层一层地搜索。
从树根开始，严格按照层次来访问节点。
BFS 过程中也可以顺便求出各个节点的深度和父亲节点。

BFS - 宽度优先搜索



比如在上图，如果我们以 5 为根，就可以有下面这个 BFS 遍历顺序：

$$5, 1, 7, 6, 2, 4, 3$$

同一棵树有多种不同不同的 BFS 遍历顺序，比如上图还可以是：

$$5, 6, 7, 1, 2, 3, 4$$

这是因为我们只规定了宽度优先，而兄弟结点的先后遍历是随意的。

BFS - BFS 序

类似的，按照 BFS 遍历顺序访问的结点序列为 BFS 序。
不过这个实际应用意义不大，这里就不多提了。

BFS - 常见代码写法

```
void bfs(int st) {
  queue<int> q; // 队列
  vector<int> vis(n); // 访问标记
  q.push(st); // 放入起点
  vis[st] = true; // 标记起点走过
  while(q.size()) {
      int now = q.front();
      q.pop;
      for(auto &x : g[now]) {
          if(vis[x])continue;
          // 标记新的结点，并放入队列
          vis[x] = true;
          q.push(x);
      }
  }
}
```

树形 DP

树形 DP

树形 DP，通常是以子树为单位进行转移，把一棵子树的信息都维护在该子树的根上。
其核心在于：

- 分而治之：将大问题分解为若干个规模更小的子问题，通过递归求解这些子问题。
- 深度优先遍历（DFS）：通常使用 DFS 来处理子树，并利用后序遍历的特性合并子问题的结果。
- 状态定义：状态通常表示为与树上某个节点相关的数据，确保在转移一个节点时，其子节点的 DP 值已经被更新好。

树形 DP 选讲 - 树上选点问题

洛谷 P1352 - 没有上司的舞会

给定一棵树，每个结点有给定的权值，可以选定一些结点，使这些结点两两不相邻，求可以选出的最大权值。

树形 DP 选讲 - 树上选点问题

记 $dp[i][0/1]$ 为结点 i 选（1）或不选（0）时，以结点 i 为根的子树的最大权值。
对于结点 u，结点 v 是结点 u 的子结点，有如下转移：

- $dp[u][0] = dp[u][0] + max(dp[v][0], dp[v][1])$，因为此时 u 不选，v 选不选都是符合题意的，用更大的来更新即可。
- $dp[u][1] = dp[u][1] + dp[v][0]$，因为此时 u 选了，所以 v 一定不能选。

最后答案就是 $max(dp[rt][0], dp[rt][1])$，$rt$ 为遍历时选取的整棵树的根，通常选取一号结点作为根。
时间复杂度 $O(n)$。

树形 DP 选讲 - 树上选点问题

蓝桥杯 2025 国赛 A 组 - 树

给定一棵树，可以选定一些结点（至少一个），使得选出的结点两两距离大于 2，问有多少种选法。

树形 DP 选讲 - 树上选点问题

类似于上面那个题，但此处距离限制更大。
但还是从树形 DP 的角度来看，如何从子结点向父结点转移。
观察可以发现，同一个结点的所有子结点，两两之间的距离都是 2，因此，对于一个结点的所有子结点，最多只能选取一个。
因此，我们可以把状态扩展一个维度，表示子结点是否存在一个结点被选取，定义为 $dp[i][j][k]$，j 表示结点 i 的选取与否，k 表示结点 i 是否存在一个被选取的子结点。

树形 DP 选讲 - 树上选点问题

对于结点 u，结点 v 是结点 u 的子结点，有如下转移：

- $dp[u][1][1] = 0$，因为此状态不合法。
- $dp[u][1][0] = \prod_{j \in g[u]} dp[j][0][0]$，因为此状态中，子结点均不选取，并且要让结点 u 可选，子结点的子结点也不可选，否则距离就会等于 2。
- $dp[u][0][0] = \prod_{j \in g[u]} dp[j][0][1] + dp[j][0][0]$，因为此状态中，结点 u 和子结点均不选取，此时子结点的子结点无论选不选，距离都一定大于 2。
- $dp[u][0][1] += dp[v][1][0] \times \prod_{j \in g[u] \wedge j \neq v} (dp[j][0][0] + dp[j][0][1])$，因为子结点只能选取一个，所以此时的转移表示的是，子结点 v 选取，并且其它兄弟节点均不选取。

最后答案就是 $dp[rt][0][1] + dp[rt][0][0] + dp[rt][1][0] - 1$，之所以 -1，是因为要减掉所有都不选的情况。

树形 DP 选讲 - 树上背包

树上背包

有 n 门课程，每一门课程有一个学分值，同时，对于每一门课，要么有且仅有一门直接先修课，要么没有先修课（并且保证先修课的关系不会成环），现在要选择 m 门课，请问可以获取的最大学分数是多少？

树形 DP 选讲 - 树上背包

定义 $dp[i][j]$ 表示结点 i 为根的子树中选择了 j 门课的最大学分值。
对于结点 u，结点 v 是结点 u 的子结点，有如下转移：

- 对于每一个结点，有一个初始状态：$dp[u][1] = a[u]$，代表只选自己。
- 从 m 到 1 枚举 k：结点 u 代表的子树中选取的课程的数量。
- 然后从 $k-1$ 到 0 枚举结点 v 代表的子树中选取的课程的数量。
- 那么有转移方程：
  $dp[u][k] = max(dp[u][k], dp[u][k-j] + dp[v][j])$。

和前面的两个题目不同，这是一个合并式的 DP，对于每一棵子树，需要考虑其它所有子树共同对父结点的影响。
也就是类似背包 DP 的思想，我们把它转化成一个前缀的问题，处理第 i 棵子树的转移时，要考虑前 $i-1$ 棵子树共同的影响，所以我们要把前 $i-1$ 棵子树的信息都合并到父结点上，在后续转移时，让父结点也参与状态转移。
这时候，就只需要枚举当前子树和父结点的状态组合用于转移了。

Intro

树基础

树的遍历

树形 DP

树上三大件

重链剖分

最近公共祖先 (LCA)

树上路径/区间问题

静态树上路径/区间问题

动态树上路径/区间问题

树上启发式合并

树上其它算法

树形 DP 选讲 - 换根 DP

刚刚的问题都有一个特点，不管选哪个结点作为根结点进行 DP，都不会影响最后的结果，那如果要求的信息会随着根节点更换而改变呢?

例如子子树结点深度和、点权和等。

如果要暴力求解出最优解，则我们可以枚举所有的节点为根，然后分别跑一次搜索，这样的时间复杂度会达到 $O(n^2)$，显然不可接受。

树形 DP 选讲 - 换根 DP

换根 DP，又称为二次扫描，是一种通过两次 DFS，可以在 $O(n)$ 的时间复杂度求出上面描述的这种问题的一种 DP 方法。
相比起一般的树形 DP，它有这些特点：

- 以树上的不同点作为根，其解不同。
- 不能单求某点的信息，需要求解每个节点的信息。
- 无法通过一次搜索完成答案的求解，因为一次搜索只能得到一个节点的答案。

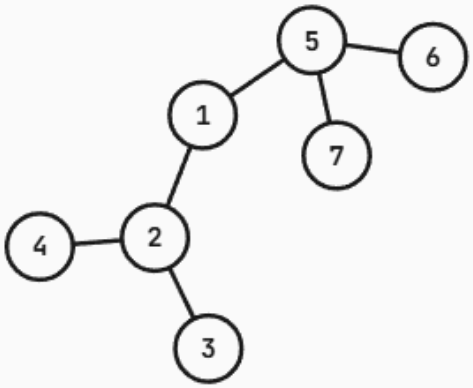
树形 DP 选讲 - 换根 DP

换根 DP 的核心如下：

- 先选定一个根，把无根树问题转化为有根树问题。
- 第一次 DFS 求出在固定某个结点为根的情况下，各子树的信息。
- 第二次 DFS，进行换根 DP，对于每一个结点，通过第一次 DFS 预处理出的子树的信息，和特殊手段维护的父子树的信息（这里的父子树，指的就是，切断该节点和父结点的边后，父结点所在的连通块），来求出该结点作为根时，该信息的值。

树形 DP 选讲 - 换根 DP

例如，对于下图，如果以 5 为根进行第一次 DFS，那么对于结点 1 而言，$\{5, 6, 7\}$ 就是它的父子树。



树形 DP 选讲 - 换根 DP

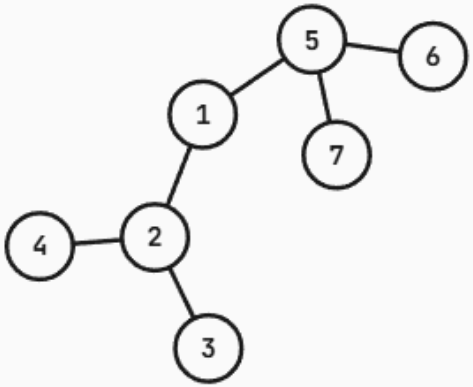
P3478 [POI 2008] STA-Station

给定一棵 n 个点的树，请求出一个结点，使得以这个结点为根时，所有结点的深度之和最大。一个结点的深度之定义为该节点到根的简单路径上边的数量。

树形 DP 选讲 - 换根 DP

知识扩展

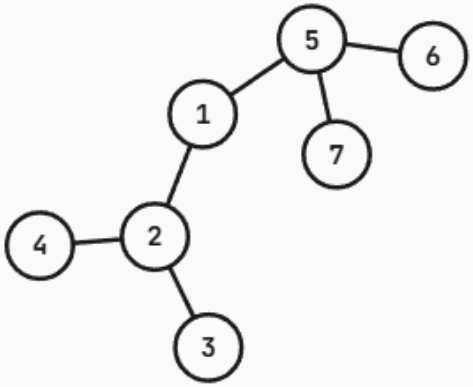
对于一棵树，它的所有结点的深度之和 = 除了它自己之外它的每棵子树的大小的和。



树形 DP 选讲 - 换根 DP

按照换根 DP 的流程，可以得到如下做法：

- 第一遍 DFS：以某个结点为根处理出每棵子树的子树大小，以及每棵子树除子树的根外所有其余结点的子树大小和。
- 第二遍 DFS：开始进行换根 DP，维护一下父子树的子树大小和，然后往各个子树走，加上新增的一棵子树的贡献加到父子树上。



树形 DP 选讲 - 换根 DP

```
int sz[N], sumsz[N];
// sz: 子树大小
// sumsz: 子树中除子树的根外所有结点的子树大小和

// 第一次 DFS
void dfs1(int u, int pre) {
  sz[u] = 1;
  for(auto &v : g[u]) {
    if(v == pre)continue;
    dfs1(v, u);
    sz[u] += sz[v];
    sumsz[u] += sumsz[v] + sz[v];
  }
}
```

树形 DP 选讲 - 换根 DP

```
// 第二次 DFS, 换根DP
int mx = 0, ans = 0;
void dfs2(int u, int pre, int up) {
  int sum = 0;
  for(auto &v : g[u]) {
    if(v == pre) sum += up;
    else sum += sumsz[v];
  }
}
if(sum > mx) ans = u, mx = sum;
for(auto &v : g[u]) {
  if(v == pre)continue;
  dfs2(v, u, sum - sumsz[x] + n - sz[v]);
}
```

树上三大件

树的直径

树的直径

树上任意两节点之间最长的简单路径即为树的直径。

树的直径模板题

给定一棵 n 个点的树，请求出该树的直径长度。

树的直径 - 解法 1：两遍 DFS

定理

在一棵树上，如果没有负边权，从任意节点 y 开始进行一次 DFS，到达的距离其最远的结点 z 必为直径的一端。

因此，只需要从任意一点 x 出发 DFS，找到最远的结点 y，再从这个结点 y 出发 DFS，找到最远的结点 z，y 和 z 形成的路径长度就是这棵树的直径。
局限性：对于带权图，不能有负边权。

树的直径 - 解法 2：树形 DP

- 思路 1：记录一下每个点 u 的子树中，从结点 u 出发向下最长和次长的两条路径，分别记为 $d1[u]$ 和 $d2[u]$，那么树的直径 $d_{max} = max(d1[u] + d2[u])$。
- 思路 2：记 $d[u]$ 为以结点 u 出发向下的最长路径。
  对于结点 u，结点 v 是结点 u 的子结点，有如下求法：
  在把 $d[v]$ 合并到 $d[u]$ 之前，
  $d_{max} = max(d_{max}, d[u] + d[v] + w(u, v))$，$w(u, v)$ 为结点 u 和 v 之间的边权。
  然后再通过 $d[u] = max(d[u], d[v] + w(u, v))$ 把子树 v 的状态值合并到子树 u 上。

树的直径 - 性质

性质

若树上所有边边权均为正，则树的所有直径中点重合。

树的直径 - 拓展题

P12534 [XJTUPC 2025] 奥日

给定一棵结点数为 $n(\leq 2*10^5)$ 的树，现在让你在树上构造出一条路径（不一定是简单路径），路径走过的结点数为 x（每走到一个结点就会对 $x+1$，无论是否第一次走到），路径上不同的结点的数量是 y，要求 $x-y\leq k$，要求构造出的路径最大化 y。

树的直径 - 拓展题

其实翻译一下题目意思就是：选出的路径上所有点被重复走的次数 $\leq k$。
首先，如果不走回头路，那么 $x = y$，但是这样的话没有用到 k，因此我们看如何利用 k 来最大化 y。
不走回头路，也就是选到了一条树上的简单路径，从一端走到另一端。
那我们先选一条简单路径出来，作为主路，起点是路径一端，终点是路径另一端。
那如果我们在路上往这条路径的分支走呢?
我们发现，往分支走后，要回到主路，就必然往回走，而每往分支多走一个没走过的结点，就必然会导致且仅导致它前一个结点被重复走一次。
所以，实际上能往分支走的结点数，就是 k。

树的直径 - 拓展题

那么思路就很简单了，我们选出一条树上的简单路径，作为主路，然后贪心地往分支走，能走则走，不够走了就往回。
这时候，我们一定能保证 k 被充分利用了，那么，要让答案尽可能大，就要让主路尽可能长，那么主路的选取也很明显了，就是树的直径。
所以，只需要找出树的直径的一端作为起点，然后贪心地往分支走即可。
由于树一定是连通的，所以有多条树的直径时，随便选一条都可以。

树的中心

树的中心

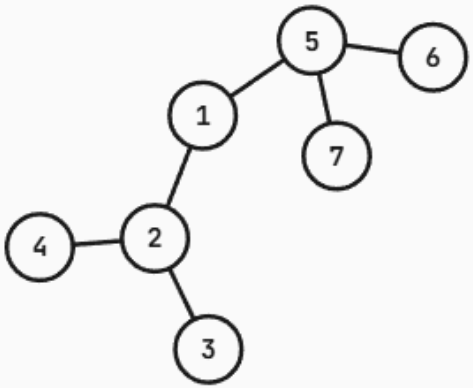
在树中，如果节点 x 作为根节点时，从 x 出发的最长链最短，那么称 x 为这棵树的中心。

树的中心 - 解法：换根 DP

应该很明显可以看出来是一个换根 DP 吧，也就是求，当每个点作为根节点的时候，这棵树的所有结点深度最大值，不过，由于这里需要的是深度的最值，不像结点数那样具有可减性，因此需要用其他手段来传递父子树的信息。
借此，顺便补充一下换根 DP 的另外两种常见处理父子树的方式。

树的中心 - 换根 DP 解法补充 1：维护最大值和次大值

遍历每个结点的以各个子结点（包括父结点，通过维护的父子树信息获取）为根的子树的最大深度，向下 DFS 的时候，如果该结点是最大值，那么传递的父子树的最大深度就应该是次大值和父子树最大深度的较大值 +1，否则，就是最大值和父子树最大深度的较大值 +1。



树的中心 - 换根 DP 解法补充 2：维护前后缀法

此方法可以说是换根 DP 的万能法，不管维护的信息是否具有可减性，其实都可以用这种方法来解。
对每个结点的子结点（包括父结点，也就是父子树的信息）维护一个前缀和一个后缀，向下 DFS 的时候，通过不包含当前子结点的前缀和不包含当前子结点的后缀来获取要传递的父子树的信息。
比如这里，就应该是维护一下以各个子结点为根的子树的最大深度的前缀最大值和后缀最大值，这样的话，就可以通过前后缀，获取到对于每个子结点而言，不包含当前结点的最大深度的最大值，也就是需要传递的父子树的信息。
由于这里的前后缀在递归的每一层都需要开，所以需要使用 vector 来开前后缀，否则会空间炸掉。

树的中心 - 性质

性质

若树上所有边边权均为正，则树的所有直径的中点（当直径上的点的数量为偶数时，这里的中点允许多个，但至多两个）重合，并且重合于树的中心。

树的重心

树的重心

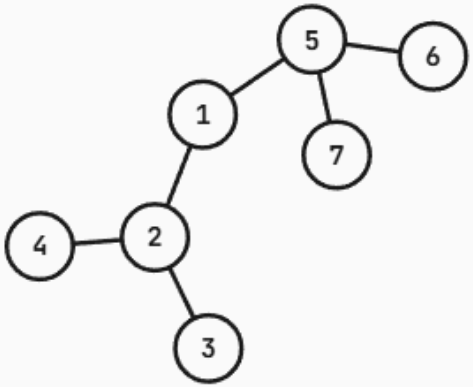
如果在树中选择某个结点并删除，这棵树将被分为若干棵子树，统计子树结点数并记录最大值。对于树上每个结点都这么操作，使此最大值取到最小的节点被称为整个树的重心。

树的重心模板题

给定一棵 n 个点的树，请求出该树的重心。

树的重心 - 解法：换根 DP

- 第一遍 DFS：人为确定一个点作为根，求出各个子树的大小。
- 第二遍 DFS：进行换根 DP，对于每个结点，其下方的子树的大小已经在第一次 DFS 求出，其父子树的大小，显然具有可减性，可以通过结点总数减掉当前结点为根的子树的大小来求出，这样，就可以知道当前结点删除后每棵子树的结点数的最大值。



树的重心 - 性质

1. 树的重心至多有两个，且这两个重心相邻。
2. 以树的重心为根时，所有子树的大小都不超过整棵树大小的一半。
3. 树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。
4. 把两棵树通过一条边相连得到一棵新的树，那么新的树的重心在连接原来两棵树的重心的路径上。
5. 在一棵树上添加或删除一个叶子，那么它的重心最多只移动一条边的距离。

树的重心 - 拓展题

CF685B - **求一棵有根树的所有子树的重心**

给定一棵有根树，求出每一棵子树（有根树意义下且包含整颗树本身）的重心是哪一个节点。(对于每一棵子树，如果有多个解，可以输出任意一个)

树的重心 - 拓展题

这个题考察的是对性质的利用。
我们可以从叶子结点开始，自底向上求答案。
首先，对于叶子结点，重心一定是自己，因为以其为根的子树只有自己。
然后我们考虑如何用性质，对每个结点，利用其子结点为根的子树的重心，递推当前结点为根的子树的重心。

树的重心 - 拓展题

首先是性质 4：把两棵树通过一条边相连得到一棵新的树，那么新的树的重心在连接原来两棵树的重心的路径上。
而对于每一个结点的子树，可以看作是它的各个子结点的子树连接到当前结点形成的，因此，归纳一下可得：对于一棵以点 u 为根的子树，其重心一定在所有以 u 的直接子节点为根的子树的重心到点 u 的路径上。
因此，当我们自底向上求解转移的时候，对于每一棵子树而言，转移时，重心的深度一定是单调不降的，因此只需要暴力地从子树已经求得的重心出发往上跳结点即可。

树的重心 - 拓展题

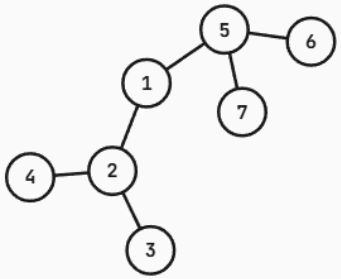
那我们又如何判断某个点是不是当前求的子树的重心呢?
这就要用到性质 2 了：以树的重心为根时，所有子树的大小都不超过整棵树大小的一半。
因此，我们只需要判断当前跳到的结点，以它为所求子树的根时，是否所有子树的大小都不超过整棵树大小的一半即可。
处理方法和换根 DP 一致，预处理出每个结点的最大子树大小，记为 $mx[i]$，假设所求子树的根节点为 u，当前跳到的结点为 p，则以 p 为所求子树的根时，最大的子树大小为 $max(mx[p], sz[u] - sz[p])$，$sz$ 为子树大小。
时间复杂度 $O(n)$。

最近公共祖先（LCA)

最近公共祖先

最近公共祖先

顾名思义，也就是两个节点共同的祖先结点中，离根最远的那个，简称为 LCA。



比如上图中，$LCA(3,4)=2, LCA(4,7)=5, LCA(4,5)=5$。
注意：在找最近公共祖先时，自己也可以看作是自己的祖先结点，这和我们前面对祖先结点的定义有些许差异。

最近公共祖先 - 性质

1. u 是 v 的祖先，当且仅当 $LCA(u, v) = u$。
2. 若 u 不是 v 的祖先，并且 v 不是 u 的祖先，那么 u, v 分别处于 $LCA(u, v)$ 的两棵不同子树中。
3. 设 S 是一个点集，在 DFS 遍历过程中，$LCA(S)$ 会在 S 中所有结点之前被遍历。
4. 两个点集 A, B，有 $LCA(A \cup B) = LCA(LCA(A), LCA(B))$。
5. 两个点的最近公共祖先必在两点间的最短路上，并且是路径上深度最低的点。
6. $d(u, v) = h(u) + h(v) - 2 * h(LCA(u, v))$，$d$ 是树上两点间的距离，h 是结点的深度。

最近公共祖先

我们可以通过倍增法，来求出对于每一个结点，它的 $1, 2, 4, 8, \ldots, 2^x$ 级祖先。
但我们只能直接获取到一级祖先，也就是父亲结点，我们如何快速获取到每个结点的 $2, 4, 8, \ldots, 2^x$ 级祖先呢?

最近公共祖先

想一下，小时候爸爸妈妈是怎么教你认长辈的？“爸爸的爸爸叫爷爷”！我们就是通过这种方式，来获取到一个结点的 2^x 级祖先。
结点 u 的 1 级祖先的 1 级祖先，也就是结点 u 的二级祖先。
结点 u 的 2 级祖先的 2 级祖先，也就是结点 u 的四级祖先。
以此类推。我们自顶向下处理这个信息，那么，当处理到 u 时，u 的所有祖先的祖先信息都已经处理完了，那么我们就可以依次获得 u 的 $1, 2, 4, 8, \ldots, 2^x$ 级祖先信息了。

最近公共祖先

预处理每个结点的 2^x 祖先的代码：

```
int fa[N][30]; // fa[u][j] 代表 u 的 2^j 级祖先

void dfs(int u, int pre) {
    fa[u][0] = pre;

    for(int k = 1;k < 30;k ++) { // 从小到大枚举
        fa[u][k] = fa[fa[u][k - 1]][k - 1];
    }

    for(auto &v : g[u]) {
        if(v == pre)continue;
        dfs(v, u);
    }
}
```

最近公共祖先

那么问题来了，为什么要维护二的次方的祖先信息呢?

最近公共祖先

我们知道，任何数都有其二进制表达，每一位要么是 0，要么是 1。也就是说，任何数都可以拆成一系列 x 不同的 2^x 相加。
那么，我们就可以通过预处理的结点 u 的 2^x 级祖先，在 $O(logn)$ 的时间复杂度，到达任何结点 u 的祖先结点，这种方法可以称其为倍增爬树。

最近公共祖先

```
int get(int u, int d) { // 获取结点 u 的 d 级祖先
    for(int i = 0;i < 30;i ++) { // 枚举顺序无所谓，因为仅仅是一系列 2^x 相加，顺序无关
        if(d >> i & 1) {
            u = fa[u][i];
        }
    }
    return u;
}
```

最近公共祖先

如果对于一个结点 u，它的 $\geq d+1$ 级祖先结点都不满足一个条件 $cond$，并且它的 $\leq d$ 级祖先都满足一个条件 $cond$，但我们并不知道 d 是多少，我们要如何求出结点 u 的 d 级祖先呢？也就是说，类似于二分，我们想找到这个临界点。

最近公共祖先

依然使用倍增，只不过，这时候我们要从大到小枚举了。
因为，对于一个结点 u，按照此枚举顺序，如果 $fa[u][k-1]$ 满足条件 $cond$，那么 $fa[fa[u][k-1]][k-1]$ 就一定不满足条件 $cond$，否则，在枚举到 k 的时候，就应该有 $u = fa[u][k]$ 的跳转。
按照这个规则跳下去，我们就一定能找到 u 的 d 级祖先，其实和倍增爬树是一个道理，只是这里无需知道距离，这种方法可以称其为临界条件爬树。

最近公共祖先

```
int get(int u) {
    for(int i = 29;i >= 0;i --) {
        if(cond(fa[u][i]) == true) { // 如果这个祖先节点条件满足，就跳上去
            u = fa[u][i];
        }
    }
    return u;
}
```

最近公共祖先

了解了这个过后，我们再来看如何这个预处理的倍增祖先数组，来求两点的 LCA。
假如 u 和 v 在深度相同，那么问题是不是就可以转化为：找到一个 d 级祖先，使得 u 和 v 的 $\geq d$ 级祖先都相同，$< d$ 级祖先都不相同。
那如果 u 和 v 不在同一层呢？也很简单，我们先把更深的那个跳到同一层不就行了，此处可以获取二者深度差值用倍增爬树，也可以使用临界条件爬树。
跳到同一层后，再把两个结点一起向上跳，最后跳到的结点的父结点，就是二者的最近公共祖先。
时间复杂度 $O(logn)$。

最近公共祖先

```
int lca(int u, int v) {
    if(dep[u] < dep[v]) { // dep是结点深度
        swap(u, v);
    }

    // 深结点跳到浅结点同层
    for(int i = 29;i >= 0;i --) {
        if(dep[fa[u][i]] < dep[v])continue;
        u = fa[u][i];
    }
    // 已经共点，说明已经找到lca了，直接返回
    if(u == v)return u;

    // 一起往上跳
    for(int i = 29;i >= 0;i --) {
        if(fa[u][i] == fa[v][i])continue;
        u = fa[u][i];
        v = fa[v][i];
    }
    return fa[u][0];
}
```

最近公共祖先

扩展：多结点 LCA

结合 LCA 的性质可知，当给出一组结点 $x_1, x_2, \ldots, x_n (dfn_{x_1} < dfn_{x_2} < \ldots < dfn_{x_n})$ 时，$LCA(x_1, x_2, \ldots, x_n) = LCA(x_1, x_n)$，证明方法不唯一，可以自行尝试证明。因此，多结点 LCA 就可以转化为两点 LCA 来 $O(1)$ 查询。

Thank you!

What you want is what you love
so fight for it!